

Control Adaptativo por Self-Tuning Regulator (STR) del Péndulo Invertido en Tiempo Discreto

Memoria Técnica

Lucas Gaggino
Facultad de Ingeniería - UBA
86.20 Control Adaptativo

15 de julio de 2026

Índice general

1.	Introducción	2
2.	Marco Teórico: el Self-Tuning Regulator	2
2.1.	Control adaptativo y self-tuning	2
2.2.	Modelo de la planta y ley de control RST	2
2.3.	Diseño por colocación de polos y ecuación Diofantina	2
2.4.	Rechazo de perturbaciones	3
2.5.	Estimación recursiva (RLS con olvido)	3
2.6.	Algoritmo indirecto	3
3.	Sistema: el Péndulo Invertido	3
3.1.	Modelo no lineal	3
3.2.	Linealización	4
3.3.	Discretización ZOH analítica y modelo ARX	4
4.	Diseño del Regulador STR (paso a paso)	4
4.1.	Estructura elegida	4
4.2.	Elección de los polos de lazo cerrado	4
4.3.	Grados y causalidad	5
4.4.	Resolución de la ecuación Diofantina (matriz de Sylvester)	5
4.5.	Prealimentación (ganancia estática unitaria)	5
4.6.	Diagrama de bloques	5
5.	Implementación	5
5.1.	Estimador RLS	6
5.2.	Ecuación Diofantina (Sylvester) y diseño RST	6
5.3.	Controlador RST	6
5.4.	Bucle adaptativo	6
6.	Resultados y Validación	7
7.	Análisis de estabilidad y convergencia; robustez	10
7.1.	Verificación del diseño	10
7.2.	Por qué el regulador no converge (modo básico)	10
7.3.	Mejoras de robustez	10
7.4.	Resultados comparativos	11
7.5.	Demostración con planta distinta a priori	13
7.6.	Suite de validación	15
8.	Discusión	15
9.	Conclusiones	16

1. Introducción

Este trabajo implementa y valida un controlador adaptativo del tipo *Self-Tuning Regulator* (STR) para estabilizar el péndulo invertido en su posición vertical. El objetivo del trabajo práctico es:

1. Implementar un controlador para la planta.
2. Evaluar su comportamiento por colocación de polos.
3. Implementar un mecanismo de autoajuste.
4. Validar el funcionamiento ante distintas condiciones.

En este proyecto el diseño del regulador se realiza enteramente en tiempo discreto. La planta nominal se discretiza una única vez mediante ZOH analítico; a partir de allí, la identificación y el diseño del regulador son íntegramente discretos: la planta se describe por su modelo ARX y el controlador se obtiene resolviendo la ecuación Diofantina en el operador retardo q^{-1} , **sin transformar el controlador nuevamente a tiempo continuo** (p. ej. por Tustin), siguiendo el método STR.

La estructura elegida es un **STR adaptativo indirecto**: en línea se estiman los polinomios $A(q)$ y $B(q)$ de la planta por mínimos cuadrados recursivos (RLS) y, por *certainty equivalence*, se rediseña el regulador resolviendo la ecuación Diofantina $AR + BS = A_{lc}$.

2. Marco Teórico: el Self-Tuning Regulator

2.1. Control adaptativo y self-tuning

Un controlador *self-tuning* converge al que se hubiese hallado con las técnicas habituales si la planta fuera conocida. Se supone conocida la *estructura* del modelo; los *parámetros* son desconocidos y se estiman on-line. Existen dos variantes:

- **Indirecto (explícito)**: se estiman $A(q)$, $B(q)$ y luego se diseña el controlador (*certainty equivalence*).
- **Directo (implícito)**: se reparametriza el problema y se estiman directamente los parámetros del controlador.

En este trabajo se usa la variante **indirecta**.

2.2. Modelo de la planta y ley de control RST

Se considera un sistema SISO lineal expresado en el operador retardo, con una perturbación v_k a la entrada:

$$A(q) y_k = B(q) (u_k + v_k), \quad (1)$$

con $A(q)$ mónico. La ley de control general de dos grados de libertad (RST) es:

$$\boxed{R(q) u_k = T(q) r_k - S(q) y_k} \quad (2)$$

donde S/R es la realimentación (*feedback*) y T/R el prealimentador (*feedforward*). La salida en lazo cerrado resulta:

$$y_k = \frac{BT}{AR + BS} r_k + \frac{BR}{AR + BS} v_k. \quad (3)$$

2.3. Diseño por colocación de polos y ecuación Diofantina

Se impone el polinomio característico deseado de lazo cerrado $A_{lc}(q)$ mediante la ecuación Diofantina:

$$A(q) R(q) + B(q) S(q) = A_{lc}(q). \quad (4)$$

Esta ecuación tiene solución si A y B no comparten factores. Se resuelve algebraicamente armando la **matriz de Sylvester** (concatenación de matrices de Toeplitz de A y B). El *controlador de mínimo orden* cumple $\deg(R) = \deg(S)$ y $\deg(S) = \deg(A) - 1$, con $\deg(A_{lc}) = 2 \deg(A) - 1$.

2.4. Rechazo de perturbaciones

Del numerador de la perturbación en (3), BR , se desprenden las reglas de diseño:

- **Perturbación de carga escalón en v :** B o R deben tener una raíz en $z = 1$. La clave está en incluir el factor $(1 - q^{-1})$ en R : como el integrador discreto es $1/(1 - q^{-1})$, al despejar u aparece su inversa y la ley adquiere *acción integral*.
- **Ruido de medición e :** para atenuar el efecto de alta frecuencia puede incorporarse el factor $z + 1$ en S . El RLS estándar **no filtra por sí mismo** el ruido de medición y, en un modelo ARX, puede producir estimaciones sesgadas (*errores en variables*); esto motiva las mejoras de robustez de la Sección 7.

2.5. Estimación recursiva (RLS con olvido)

Para un modelo ARX, con vector de parámetros $\theta = [a_1, \dots, a_n, b_1, \dots, b_m]^\top$ y regresor $\varphi(k)$, el RLS con factor de olvido $\lambda \in (0, 1]$ es:

$$\epsilon(k) = y(k) - \varphi^\top(k) \hat{\theta}(k-1), \quad (5)$$

$$K(k) = \frac{P(k-1)\varphi(k)}{\lambda + \varphi^\top(k)P(k-1)\varphi(k)}, \quad (6)$$

$$\hat{\theta}(k) = \hat{\theta}(k-1) + K(k)\epsilon(k), \quad (7)$$

$$P(k) = \frac{1}{\lambda}(I - K(k)\varphi^\top(k))P(k-1). \quad (8)$$

$\lambda \rightarrow 1$ permite seguir parámetros lentamente variables; valores menores olvidan más rápido a costa de mayor sensibilidad al ruido.

2.6. Algoritmo indirecto

1. En cada k , recolectar $\varphi(k)$ e $y(k)$.
2. Actualizar RLS $\Rightarrow \hat{A}(q), \hat{B}(q)$.
3. Resolver la Diofantina (4) $\Rightarrow R, S, T$.
4. Calcular la acción de control con (2).
5. Aplicar $u(k)$ y repetir.

No es necesario recalcular el controlador en cada paso: puede hacerse cada N pasos. Es crítico garantizar la *persistencia de excitación*.

3. Sistema: el Péndulo Invertido

3.1. Modelo no lineal

La planta “real” que se controla en la simulación es el modelo no lineal del carro-péndulo. Se integran los **cuatro** estados $[x, \dot{x}, \theta, \dot{\theta}]$ (aunque el control sólo usa θ):

$$\ddot{\theta} = \frac{g \sin \theta - \cos \theta \cdot \frac{F + m\dot{\theta}^2 \sin \theta}{M + m}}{l \left(\frac{4}{3} - \frac{m \cos^2 \theta}{M + m} \right)}, \quad (9)$$

$$\ddot{x} = \frac{F + m\dot{\theta}^2 \sin \theta}{M + m} - \frac{m\ddot{\theta} \cos \theta}{M + m}, \quad (10)$$

con $M = 1,0$ kg (carro), $m = 0,1$ kg (barra), $g = 9,81$ m/s² y $l = 0,5$ m la **distancia del pivote al centro de masa** (media longitud de la barra); el término $\frac{4}{3}$ proviene del momento de inercia de una barra uniforme. Se integra con Runge-Kutta manteniendo la acción de control constante en cada período (ZOH). Como no se controla la posición del carro, x y $\dot{x}$ no intervienen en el diseño; la dinámica angular alcanza para el control.

3.2. Linealización

Se linealiza mediante el **jacobiano** de $\ddot{\theta} = f(\theta, \dot{\theta}, F)$ evaluado en el equilibrio inestable $(\theta, \dot{\theta}, F) = (0, 0, 0)$ (desarrollo de Taylor de primer orden):

$$A_\theta = \left. \frac{\partial f}{\partial \theta} \right|_{eq}, \quad B_\theta = \left. \frac{\partial f}{\partial F} \right|_{eq}, \quad \left. \frac{\partial f}{\partial \dot{\theta}} \right|_{eq} = 0, \quad (11)$$

donde el término en $\dot{\theta}$ se anula porque el único aporte, $ml\dot{\theta}^2 \sin \theta$, tiene derivada $2ml\dot{\theta} \sin \theta = 0$ en el equilibrio. Evaluando las derivadas (equivalente a la aproximación de ángulo pequeño $\sin \theta \approx \theta$, $\cos \theta \approx 1$ y despreciando el término cuadrático $\dot{\theta}^2 \sin \theta$) se obtiene:

$$\ddot{\theta} = A_\theta \theta + B_\theta F, \quad A_\theta = \frac{3g(M+m)}{l(4M+m)} = 15,79, \quad B_\theta = \frac{-3}{l(4M+m)} = -1,4634. \quad (12)$$

El signo $B_\theta < 0$ es físicamente correcto: un empuje $F > 0$ sobre el carro reduce θ . Este signo es esencial: usar el signo opuesto produce realimentación positiva y el lazo diverge.

3.3. Discretización ZOH analítica y modelo ARX

Para $G(s) = B_\theta/(s^2 - w^2)$ con $w = \sqrt{A_\theta}$, el mantenedor de orden cero da:

$$A(z) = z^2 - 2 \cosh(wT_s) z + 1, \quad B(z) = \frac{B_\theta(\cosh(wT_s) - 1)}{A_\theta} (z + 1), \quad (13)$$

con $T_s = 0,02$ s. Numéricamente, el modelo ARX nominal es:

$$A(q) = 1 - 2,0063 q^{-1} + 1,0 q^{-2}, \quad B(q) = -2,93 \times 10^{-4} (q^{-1} + q^{-2}). \quad (14)$$

Observaciones de diseño. La planta discreta tiene un polo en $z \approx 1,083$ (*fuera* del círculo unitario, inestable) y otro en $z \approx 0,924$. El cero está en $z \approx -1$ (sobre el círculo unitario): es **no de fase mínima**, por lo que **no se lo cancela** (cancelarlo generaría oscilaciones de alta amplitud, *ringing*). Este modelo nominal se usa solo como *prior* de arranque del STR; los parámetros reales se estiman on-line.

4. Diseño del Regulador STR (paso a paso)

4.1. Estructura elegida

STR indirecto, por colocación de polos, **sin cancelación de ceros** y con **acción integral**. El factor $1 - q^{-1}$ es el operador *diferencia*; el integrador discreto es $1/(1 - q^{-1})$. Al incluir $1 - q^{-1}$ en R , al despejar u de (2) aparece su inversa y la ley de control adquiere acción integral. Para ello se diseña sobre

$$\bar{A}(q) = A(q) (1 - q^{-1}), \quad (15)$$

de grado 3, y luego $R = (1 - q^{-1}) \bar{R}$.

4.2. Elección de los polos de lazo cerrado

Se reutiliza la especificación del diseño previo ($s = -3, -3, -10$), mapeada a discreto con $z = e^{sT_s}$:

$$\{-3, -3, -10\} \rightarrow \{0,9418, 0,9418, 0,8187\}. \quad (16)$$

La Diofantina con acción integral requiere $\deg(A_{lc}) = 5$, por lo que se agregan dos polos auxiliares rápidos en $z = 0,2$ (bien amortiguados, cerca del origen) para completar el grado sin alterar la

dinámica dominante. Para mantener coherencia con la ecuación Diofantina (plantada en q^{-1}), se escribe:

$$A_{lc}(q^{-1}) = \prod_{i=1}^5 (1 - p_i q^{-1}), \quad p_i \in \{0,9418, 0,9418, 0,8187, 0,2, 0,2\}. \quad (17)$$

Sus coeficientes son los que devuelve `np.poly(polos_lc)`.

4.3. Grados y causalidad

Con $\deg(\bar{A}) = 3$ y $\deg(B) = 2$, el controlador de mínimo orden tiene $\deg(\bar{R}) = 2$ y $\deg(S) = 2$. El regulador final:

$$R(q) = (1 - q^{-1}) \bar{R}(q) \text{ (grado 3)}, \quad S(q) = s_0 + s_1 q^{-1} + s_2 q^{-2}. \quad (18)$$

4.4. Resolución de la ecuación Diofantina (matriz de Sylvester)

Se resuelve $\bar{A}(q) \bar{R}(q) + B(q) S(q) = A_{lc}(q)$ con $\bar{A} = [1, \bar{a}_1, \bar{a}_2, \bar{a}_3]$, $B = [0, b_1, b_2]$, $\bar{R} = [1, r_1, r_2]$ y $S = [s_0, s_1, s_2]$. Igualando coeficientes de q^{-1} a q^{-5} se obtiene el sistema lineal (matriz de Sylvester):

$$\begin{bmatrix} 1 & 0 & b_1 & 0 & 0 \\ \bar{a}_1 & 1 & b_2 & b_1 & 0 \\ \bar{a}_2 & \bar{a}_1 & 0 & b_2 & b_1 \\ \bar{a}_3 & \bar{a}_2 & 0 & 0 & b_2 \\ 0 & \bar{a}_3 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} r_1 \\ r_2 \\ s_0 \\ s_1 \\ s_2 \end{bmatrix} = \begin{bmatrix} \alpha_1 - \bar{a}_1 \\ \alpha_2 - \bar{a}_2 \\ \alpha_3 - \bar{a}_3 \\ \alpha_4 \\ \alpha_5 \end{bmatrix}, \quad (19)$$

donde α_i son los coeficientes de A_{lc} . Para el modelo nominal ($\bar{A} = [1, -3,0063, 3,0063, -1,0]$) la solución es:

$$R = [1, -1,2033, 0,2324, -0,0290], \quad S = [-366,77, 697,19, -331,09]. \quad (20)$$

Los valores grandes de S reflejan la alta ganancia necesaria para estabilizar un sistema inestable.

4.5. Prealimentación (ganancia estática unitaria)

Se impone $y/r|_{q^{-1}=1} = 1$, es decir $B(1)T/A_{lc}(1) = 1$, de donde

$$T = t_0 = \frac{A_{lc}(1)}{B(1)} = \frac{\sum_i \alpha_i}{b_1 + b_2} = -0,6718. \quad (21)$$

Como la referencia es $\theta = 0$, el término Tr es nulo, pero se deja general.

4.6. Diagrama de bloques

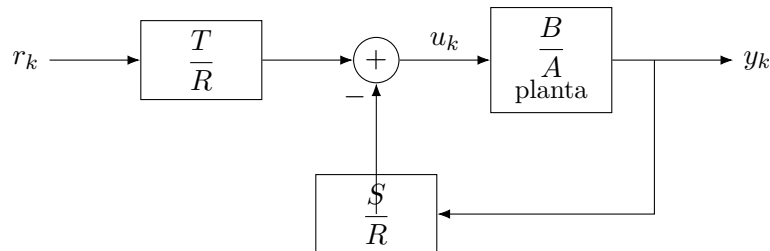


Figura 1: Estructura RST de dos grados de libertad: feedforward T/R , feedback S/R y planta B/A . La ley $Ru = Tr - Sy$ incluye el factor $(1 - q^{-1})$ en R , que introduce acción integral.

5. Implementación

El código está en `respuesta_pid_pendulo_autotunning.py`. Se resumen las piezas centrales.

5.1. Estimador RLS

```
1 def rls_step(theta, P, phi, y_meas, lambda_=0.995):
2     y_pred = phi @ theta
3     err = y_meas - y_pred
4     Pphi = P @ phi
5     K = Pphi / (lambda_ + phi @ Pphi)
6     theta = theta + K * err
7     P = (P - np.outer(K, Pphi)) / lambda_
8     return theta, P, err
```

Listing 1: Un paso de RLS con olvido.

5.2. Ecuacion Diofantina (Sylvester) y disenio RST

```
1 def solve_diophantine_sylvester(A_bar, B, Alc, deg_R, deg_S):
2     n_rows = len(Alc)
3     n_unk = deg_R + (deg_S + 1)
4     M_syl = np.zeros((n_rows, n_unk))
5     for i in range(1, deg_R + 1):
6         for k in range(len(A_bar)):
7             M_syl[i + k, i - 1] += A_bar[k]
8     off = deg_R
9     for j in range(0, deg_S + 1):
10        for k in range(len(B)):
11            M_syl[j + k, off + j] += B[k]
12    rhs = Alc.copy()
13    for k in range(len(A_bar)):
14        rhs[k] -= A_bar[k]
15    sol = np.linalg.solve(M_syl[1:1 + n_unk, :], rhs[1:1 + n_unk])
16    return np.concatenate(([1.0], sol[:deg_R])), sol[deg_R:]
```

Listing 2: Armado de la matriz de Sylvester y resolucion.

```
1 def design_rst(theta_hat, polos_lc=POLOS_LC):
2     a1, a2, b1, b2 = theta_hat
3     A = np.array([1.0, a1, a2])
4     B = np.array([0.0, b1, b2])
5     A_bar = np.convolve(A, [1.0, -1.0])
6     Alc = np.real(np.poly(polos_lc))
7     R_bar, S = solve_diophantine_sylvester(A_bar, B, Alc, deg_R=2, deg_S=2)
8     R = np.convolve([1.0, -1.0], R_bar)
9     t0 = np.sum(Alc) / (b1 + b2)
10    return R, S, t0
```

Listing 3: Diseno del regulador con accion integral y sin cancelacion.

5.3. Controlador RST

```
1 def control(self, r, y):
2     yvec = np.concatenate([y, self.y_hist])[len(self.S):]
3     s_term = float(self.S @ yvec)
4     r_term = float(self.R[1:] @ self.u_hist[:len(self.R) - 1])
5     u = (self.t0 * r - s_term - r_term) / self.R[0]
6     # ... actualizar historiales de u e y ...
7     return u
```

Listing 4: Ley recursiva $R u = T r - S y$.

5.4. Bucle adaptativo

En cada paso: (i) RLS actualiza $\hat{\theta} = [a_1, a_2, b_1, b_2]$; (ii) tras un *warmup* y cada `redesign_every` pasos se rediseña el RST; (iii) se aplica la acción de control; (iv) se integra la ODE no lineal (ZOH). Dos salvaguardas importantes:

- **Freno por baja excitación:** el RLS solo actualiza si $\|\varphi\|$ supera un umbral. Al estabilizarse el péndulo, las señales tienden a cero y sin este freno el estimador se degradaría (problema de persistencia de excitación).
- **Guarda de divergencia:** si $|\theta| > 90^\circ$ (el péndulo “cayó”) se detiene la integración.

```

1 if k >= 2:
2     phi = np.array([-y[k-1], -y[k-2], u[k-1], u[k-2]])
3     if np.linalg.norm(phi) > excite_thresh:
4         theta_hat, P, _ = rls_step(theta_hat, P, phi, y_measured, lambda_)
5
6 if k >= warmup_k and (k - warmup_k) % redesign_every == 0:
7     R_new, S_new, t0_new = design_rst(theta_hat) # certainty equivalence
8     ctrl_rst.set_params(R_new, S_new, t0_new)
9
10 u_control = ctrl_rst.control(ref[k], y_measured)

```

Listing 5: Nucleo del bucle de simulacion.

6. Resultados y Validación

Se simularon 20 s desde la condición inicial $\theta(0) = 0,1$ rad con referencia $\theta = 0$, en cuatro escenarios. En todos, el ángulo se estabiliza (nunca supera la condición inicial) y la acción de control se mantiene acotada ($|u|_{\text{máx}} \approx 37$ N). Sin perturbaciones, la estimación RLS converge al modelo real $[a_1, a_2, b_1, b_2] \approx [-2,006, 1,0, -2,9 \times 10^{-4}, -2,9 \times 10^{-4}]$. El análisis de convergencia del regulador ante ruido y perturbaciones se detalla en la Sección 7.

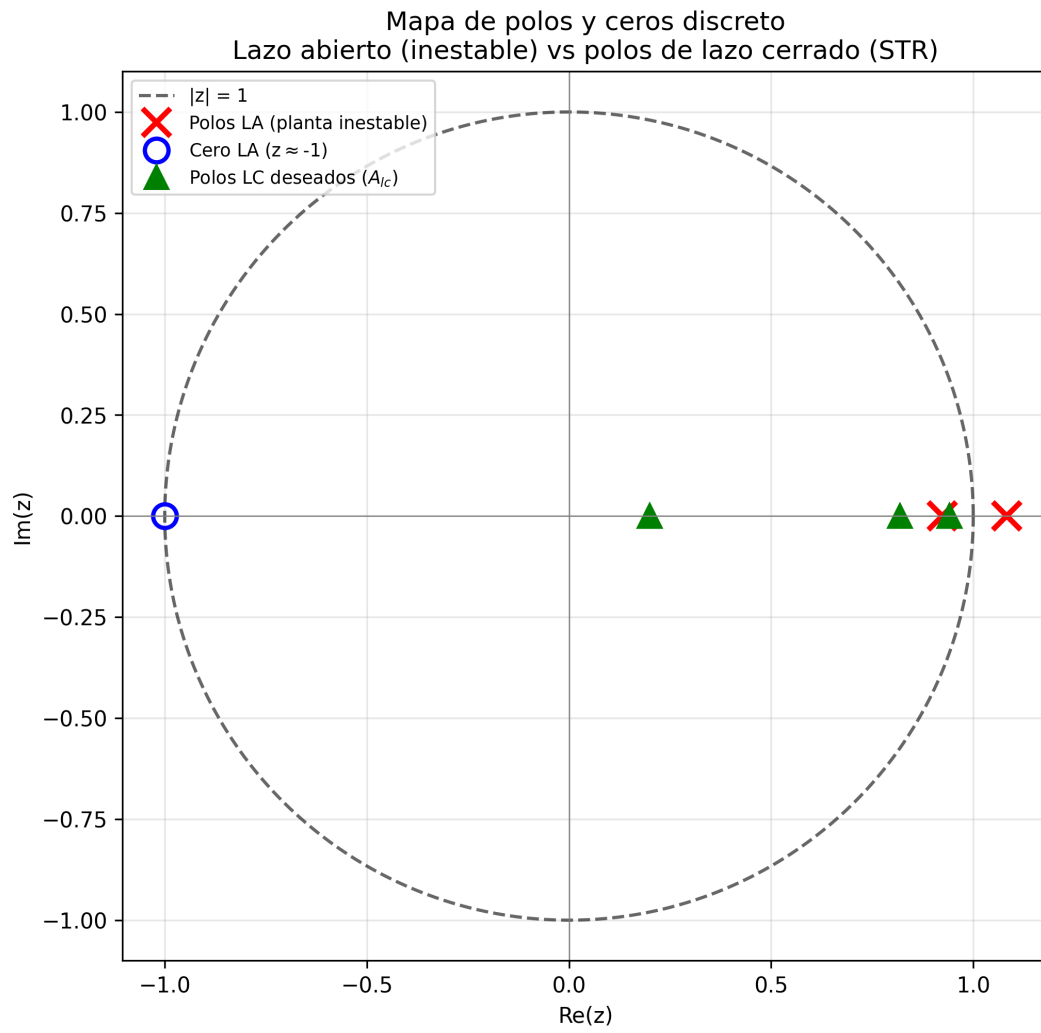
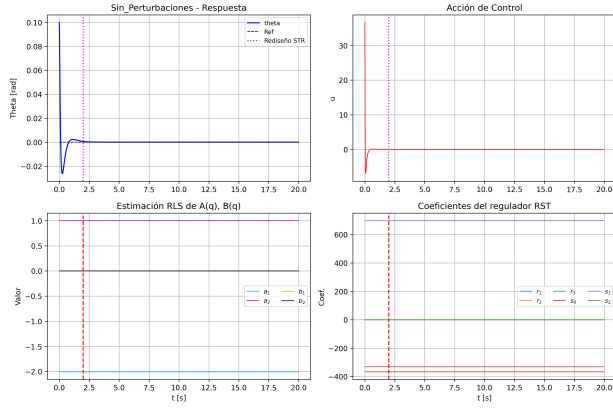
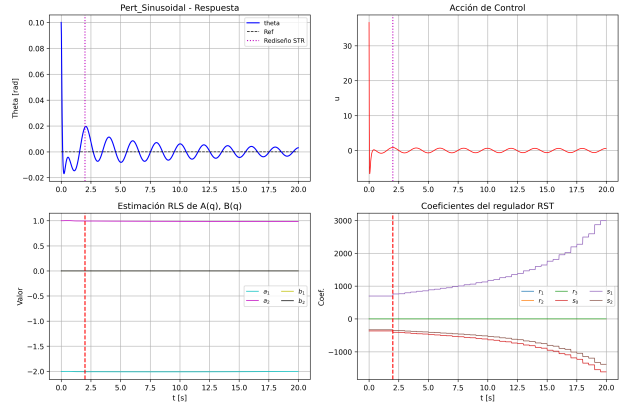


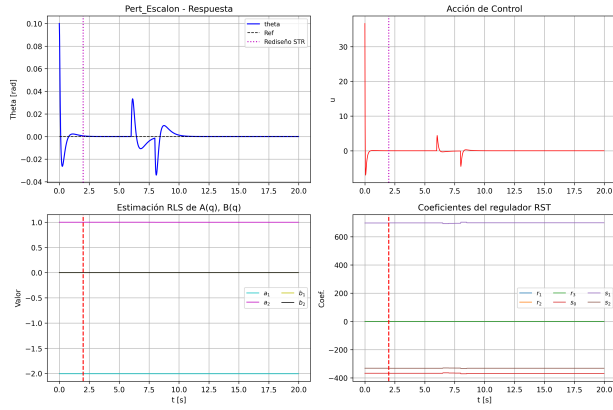
Figura 2: Mapa de polos y ceros discreto. El polo de lazo abierto fuera del círculo unitario (planta inestable) y el cero en $z \approx -1$; los polos de lazo cerrado impuestos por el STR quedan dentro del círculo unitario.



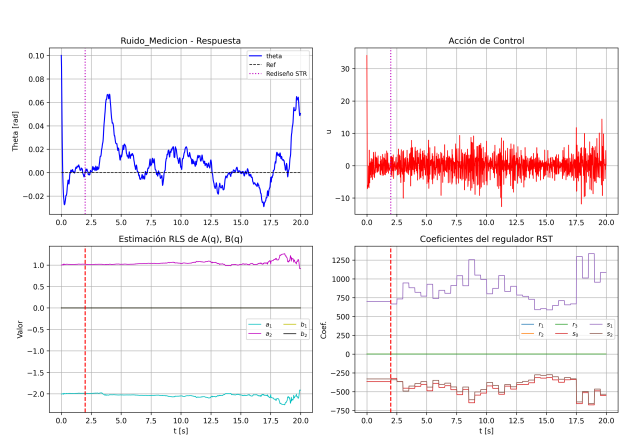
(a) Sin perturbaciones



(b) Perturbación sinusoidal (en la salida)



(c) Perturbación rectangular (pulso $t \in [6, 8]$, en la salida)



(d) Ruido de medición ($\sigma = 0,005$, sólo en y_{medida})

Figura 3: Respuesta (θ), acción de control (u), estimación RLS de $A(q), B(q)$ y coeficientes del regulador RST para los cuatro escenarios.

Puntos de inyección. Las tres perturbaciones tienen transferencias distintas y se aplican en lugares distintos: la *sinusoidal* y el *pulso rectangular* se suman a la salida verdadera θ (perturbación de *salida*, no de carga), mientras que el *ruido* se agrega exclusivamente a la salida medida y_{medida} (ruido de medición).

El máximo de $|\theta|$ está dominado por la condición inicial (0,1 rad) y por eso aporta poca información. La Tabla 1 usa métricas más representativas del efecto de las perturbaciones: el RMS y el máximo de θ después del transitorio ($t > 5$ s), el tiempo de establecimiento ($|\theta| < 0,02$), el esfuerzo de control (RMS de u) y la dispersión de s_0 en la ventana final (indicador de convergencia del regulador). Se reportan para el modo robusto (el análisis básico vs. robusto está en la Sección 7).

Cuadro 1: Métricas de desempeño (modo robusto).

Escenario	$\text{RMS}_{\theta}^{t>5}$	$ \theta _{\max}^{t>5}$	t_{est} [s]	RMS_u	$\text{std}(s_0)$
Sin perturbaciones	0,0000	0,0000	0,36	1,27	≈ 0
Perturbación sinusoidal	0,0134	0,0189	0,08	1,38	≈ 0
Pulso rectangular	0,0062	0,0344	8,24	1,33	≈ 0
Ruido de medición	0,0026	0,0072	0,36	2,75	≈ 0

El factor $(1 - q^{-1})$ en R dota al lazo de *acción integral*, lo que permite rechazar perturbaciones de tipo escalón. La perturbación de la Figura 3(c) es en rigor un *pulso rectangular* (escalón de

subida en $t = 6$ s y de bajada en $t = 8$ s): el t_{est} de 8,24 s refleja el rechazo del flanco de subida y la recuperación tras el de bajada. Como el pulso desaparece antes de alcanzar necesariamente el régimen permanente, no se lo usa para afirmar “error estacionario nulo”; esa propiedad se sigue de la acción integral ante una perturbación de escalón sostenida.

7. Análisis de estabilidad y convergencia; robustez

El ángulo θ permanece acotado en los cuatro escenarios, pero un análisis más fino revela que **los coeficientes del regulador R, S, T no convergen** con perturbación sinusoidal ni con ruido de medición. Esta sección lo diagnostica, lo corrige con una variante *robusta* y valida todo con una suite de tests.

7.1. Verificación del diseño

Con el modelo nominal, la solución de la Diofantina reproduce *exactamente* los polos deseados: las raíces de $A(q)R(q) + B(q)S(q)$ coinciden con A_{lc} ($\max |z| = 0,9418 < 1$). Los polos elegidos (mapeo e^{sT_s} de $-3, -3, -10$ más dos auxiliares en 0,2) son sensatos: reales, positivos y bien dentro del círculo unitario. El test `test_diophantine_nominal` verifica esta igualdad automáticamente.

7.2. Por qué el regulador no converge (modo básico)

La ley de control aplicada a la planta *real* usa el regulador diseñado con el modelo *estimado*. Cuando ese estimado se degrada, el regulador “persigue” el ruido. La causa raíz es que el numerador B es diminuto ($b_1 + b_2 \approx -5,9 \times 10^{-4}$) y aparece en el *denominador* del diseño ($t_0 = A_{lc}(1)/B(1)$ y la matriz de Sylvester): un error relativo en b se amplifica enormemente en R, S, T . Concretamente:

- **Perturbación sinusoidal (en la salida).** Al sumarse a θ , la senoide contamina el regresor del RLS, que reparte mal la ganancia: b_1 llega a cambiar de signo, el cero estimado salta a $z \approx +1,7$ y los coeficientes de S crecen *monótonamente* (s_1 pasa de ~ 700 a ~ 3000 , Fig. 5). El radio de polos de LC (evaluado sobre el modelo nominal) trepa de 0,942 a 0,989: el lazo sigue estable pero pierde margen de forma sostenida.
- **Ruido de medición (sólo en y_{medida}).** El RLS adapta sobre el ruido, agravado por *covariance windup* ($\lambda < 1$ con baja excitación): a_1, a_2 quedan sesgados, s_0 fluctúa $\pm 20\%$ y, en algunos rediseños, el regulador instantáneo llega a colocar polos *fuera* del círculo unitario ($\max |z| > 1$, Fig. 6). El lazo sobrevive por ser lentamente variable, pero el diseño es momentáneamente inestable.

7.3. Mejoras de robustez

Se agrega una variante robusta (`robust=True`), comparable con la básica. Cada mecanismo ataca una de las causas anteriores:

- **Zona muerta** en el RLS (contra el ruido y la perturbación de salida): no se actualiza si $|\varepsilon|$ no supera el error de ecuación esperado ($\sim \sigma \|[1, a_1, a_2]\|$), evitando que el estimador “persiga” señales que no son la dinámica de la planta.
- **Anti *windup* de covarianza** (contra el *covariance windup* del caso de ruido): se acota $\text{tr}(P)$ para que la ganancia del RLS no crezca sin límite bajo baja excitación.
- **Gate de rediseño** (contra los diseños con b mal estimado): validado contra el *prior nominal confiable* (no contra la estimación ruidosa), acepta un candidato sólo si $b_1 + b_2$ conserva signo y magnitud, $|t_0|$ y $\|S\|$ son razonables y el lazo cerrado resultante es estable. Además sólo se rediseña ante cambios significativos de $\hat{\theta}$ (congelamiento). *Limitación:* el gate sólo garantiza estabilidad respecto del *prior nominal*; no puede asegurarla frente a una planta real desconocida.

```

1 def candidato_valido(R, S, t0, theta_hat, s_norm_max, cfg, plant_ref):
2     b1, b2 = theta_hat[2], theta_hat[3]
3     if (b1 + b2) >= 0 or abs(b1 + b2) < cfg["b_min"]: # signo/magnitud de b
4         return False
5     if abs(t0) > cfg["t0_max"] or np.linalg.norm(S) > s_norm_max:
6         return False
7     poles = closed_loop_poles(plant_ref, R, S) # estabilidad vs prior
8     return np.max(np.abs(poles)) < 1.0 - cfg["pole_margin"]

```

Listing 6: Gate de aceptación del regulador candidato (robusto).

7.4. Resultados comparativos

La variante robusta hace converger R, S, T y recupera exactamente los polos de diseño ($\max |z| = 0,9418$) en los cuatro escenarios, mientras la básica deriva (Tabla 2, Figuras 4, 5 y 6).

Cuadro 2: Máximo módulo de los polos de lazo cerrado *evaluados sobre el modelo lineal nominal alrededor de $\theta = 0$* (con el regulador final) y convergencia (desvío de s_0 en la ventana final). El gate del modo robusto sólo verifica estabilidad frente a este modelo nominal, no frente a la planta real desconocida.

Escenario	Básico		Robusto	
	máx $ z $	std(s_0)	máx $ z $	std(s_0)
Sin perturbaciones	0,942	≈ 0	0,942	0
Perturbación sinusoidal	0,989	190	0,942	0
Pulso rectangular	0,957	≈ 0	0,942	0
Ruido de medición	0,995	126	0,942	0

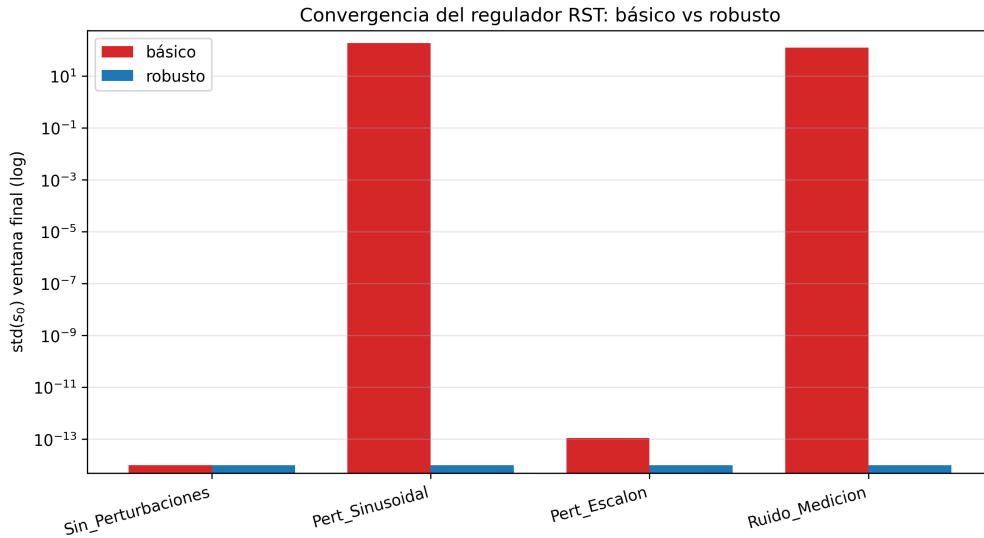


Figura 4: Desvío estándar de s_0 en la ventana final (escala log). El modo robusto (azul, prácticamente nulo) converge en todos los escenarios; el básico (rojo) no converge con perturbación sinusoidal ni con ruido.

Sinusoidal (los dos casos que más cambian). La Figura 5 detalla el caso sinusoidal. Es importante ser honestos: en régimen, el modo básico logra un RMS de θ *menor* (0,0040 vs 0,0134), porque su ganancia s_1 crece sin límite y eso mejora el rechazo de la perturbación de salida a 0,5 Hz.

Sin embargo, ese mismo crecimiento es lo que aleja los polos hacia el círculo unitario ($0,942 \rightarrow 0,989$) y hace que R, S, T nunca converjan. El robusto sacrifica algo de rechazo del rizo a cambio de coeficientes convergidos y margen de estabilidad constante.

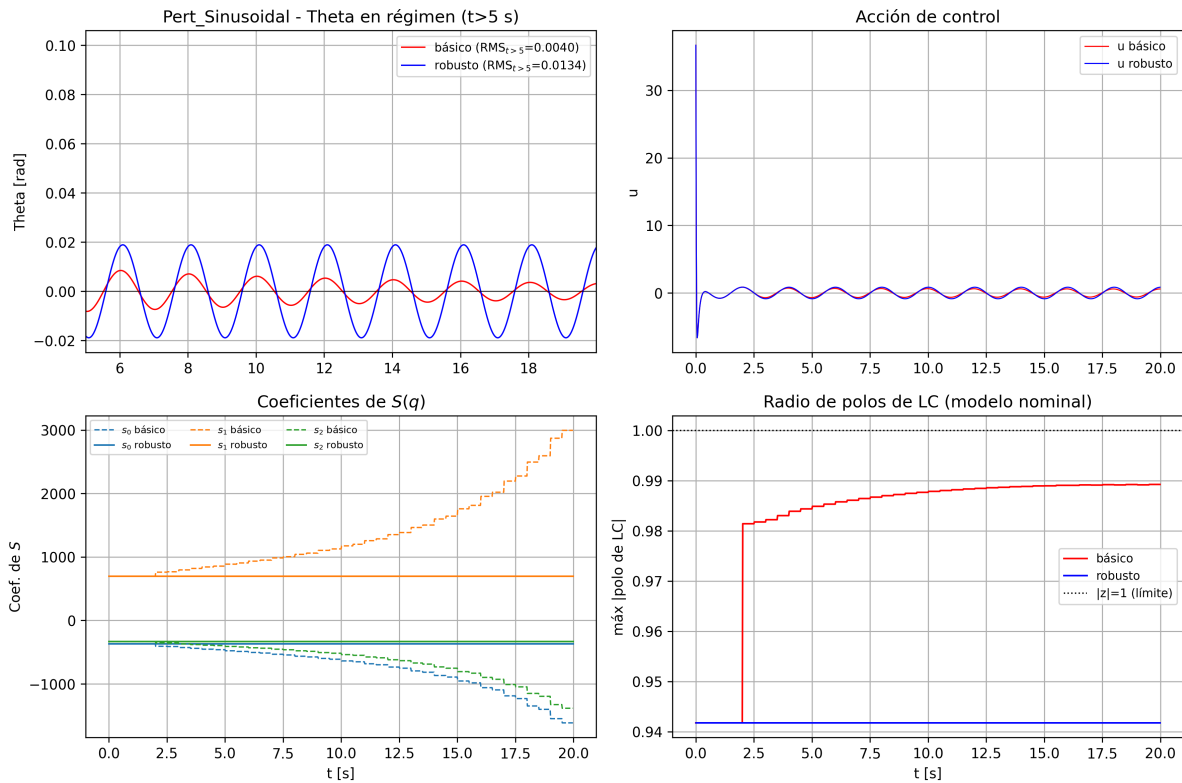


Figura 5: Perturbación sinusoidal: básico vs robusto. Arriba: θ en régimen (con RMS) y control u . Abajo: coeficientes de S (básico en trazo punteado, diverge; robusto sólido, constante) y radio de polos de LC (el básico sube a 0,989; el robusto se mantiene en 0,942).

Ruido de medición. La Figura 6 detalla el caso de ruido, donde el robusto es netamente superior: el RMS de θ en régimen baja de 0,0165 a 0,0026 (unas $6\times$), el tiempo de establecimiento pasa de ~ 20 s a 0,36 s y el radio de polos deja de superar el círculo unitario.

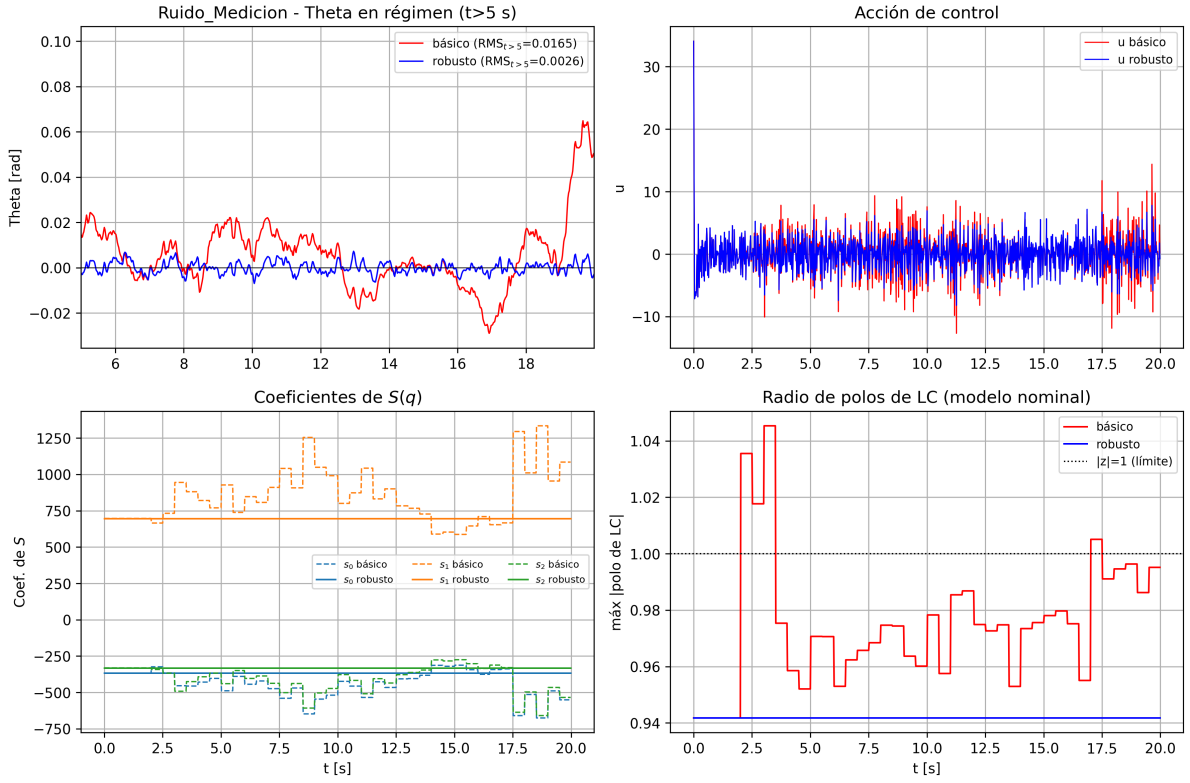


Figura 6: Ruido de medición: básico vs robusto. El regulador básico persigue el ruido (S oscila y el radio de polos supera $|z| = 1$ en varios instantes); el robusto se estabiliza en el diseño nominal y regula mucho mejor a θ .

7.5. Demostración con planta distinta a priori

En los cuatro escenarios anteriores el prior nominal coincide con la linealización de la planta, de modo que la variante robusta recupera y congela el regulador nominal. Eso muestra robustez del controlador nominal, pero no prueba por sí solo la capacidad de *autoajuste*: para ello hace falta que la planta real difiera del prior.

Caso 1: Carrito 50 % más pesado (cambia la ganancia). Con $M_{\text{real}} = 1,5 M_{\text{nominal}}$ y prior nominal, la ganancia real cambia de $b_1 + b_2 \approx -5,9 \times 10^{-4}$ a $\approx -3,9 \times 10^{-4}$. Como sin excitación la tarea de regulación pierde persistencia, se inyecta una referencia cuadrada de pequeña amplitud (dither) y se reduce la zona muerta. Los resultados (Fig. 7) confirman el autoajuste:

1. el RLS arranca del prior nominal *incorrecto*;
2. la ganancia estimada se desplaza hacia el modelo real ($b_1 + b_2 : -5,9 \rightarrow -3,9 \times 10^{-4}$);
3. el RST se rediseña: S pasa de $[-367, 697, -331]$ (nominal) a $\approx [-551, 1048, -498]$, que coincide con el diseño calculado para la planta real ($[-545, 1037, -492]$);
4. el controlador final **difiere** del nominal y el lazo permanece estable ($\max |z| < 1$ sobre la planta real, $|\theta|_{\text{máx}} = 0,10$).

Es notable que en este caso a_1, a_2 *no* se mueven. La razón es física: $a_2 \equiv 1$ es estructural (el producto de los polos discretos $e^{\pm wT_s}$ del péndulo sin fricción es 1), y $a_1 = -2 \cosh(wT_s)$ depende de $A_\theta = 3g(M + m)/(l(4M + m))$, que es casi **insensible a la masa del carro** (numerador y denominador crecen ambos con M y el cociente satura en $3g/4l$). La masa afecta esencialmente sólo la ganancia $B_\theta \propto 1/M$, es decir b . El RLS mueve exactamente lo que cambió.

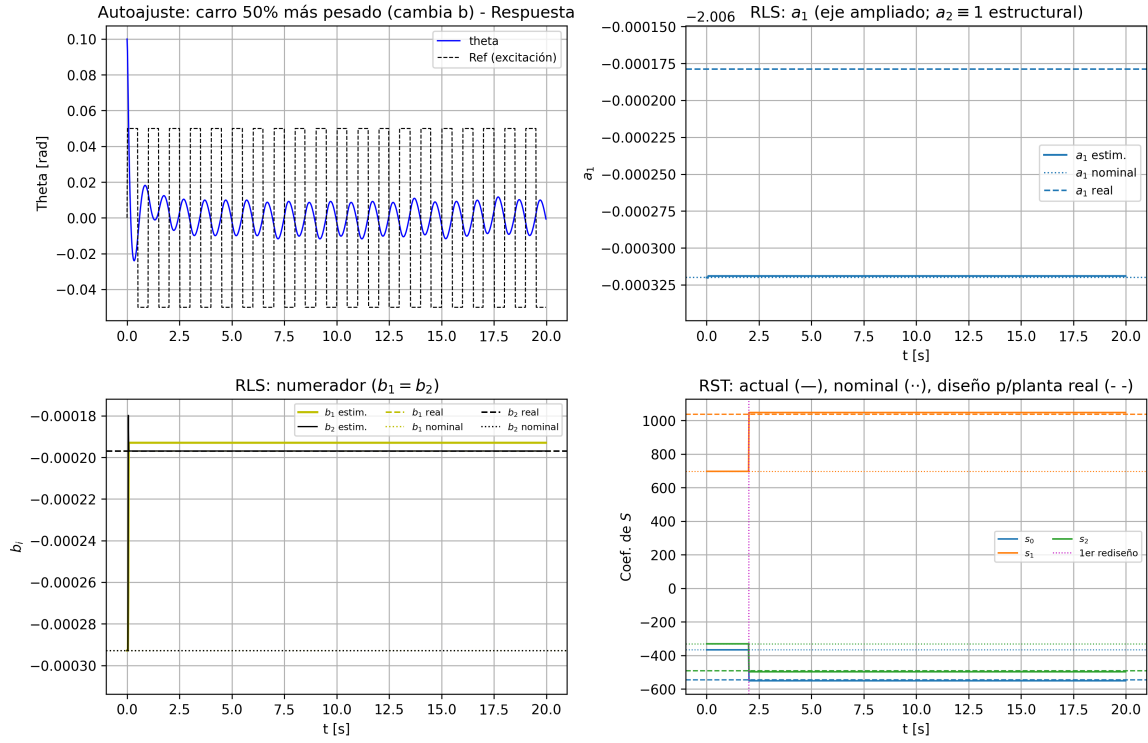


Figura 7: Autoajuste ante un carro 50% más pesado. Estimados (línea continua) parten del prior nominal (punteado) y van hacia la planta real (guiones): b converge al real; a_1 (eje ampliado) casi no cambia porque la masa apenas afecta los polos. El regulador S se rediseña hacia el diseño de la planta real.

Caso 2: barra 10% más larga (cambian los polos y la ganancia). Para mostrar además el desplazamiento de los *polos*, se alarga la barra un 10% ($l_{\text{real}} = 1,1l$), lo que sí cambia A_θ y por lo tanto a_1 (de $-2,00632$ a $-2,00575$). Como a_1 multiplica a $y \approx 0$ (péndulo regulado), está mucho peor excitado que b (que multiplica a u , grande): para identificarlo hace falta **más excitación** (dither de mayor amplitud) y **menor zona muerta**. Con esos ajustes (Fig. 8) el a_1 estimado se desplaza claramente desde el nominal hacia el real ($-2,00632 \rightarrow -2,00599$), b converge al real y S se rediseña a $\approx [-402, 766, -364]$ (diseño de la planta real), con el lazo estable.

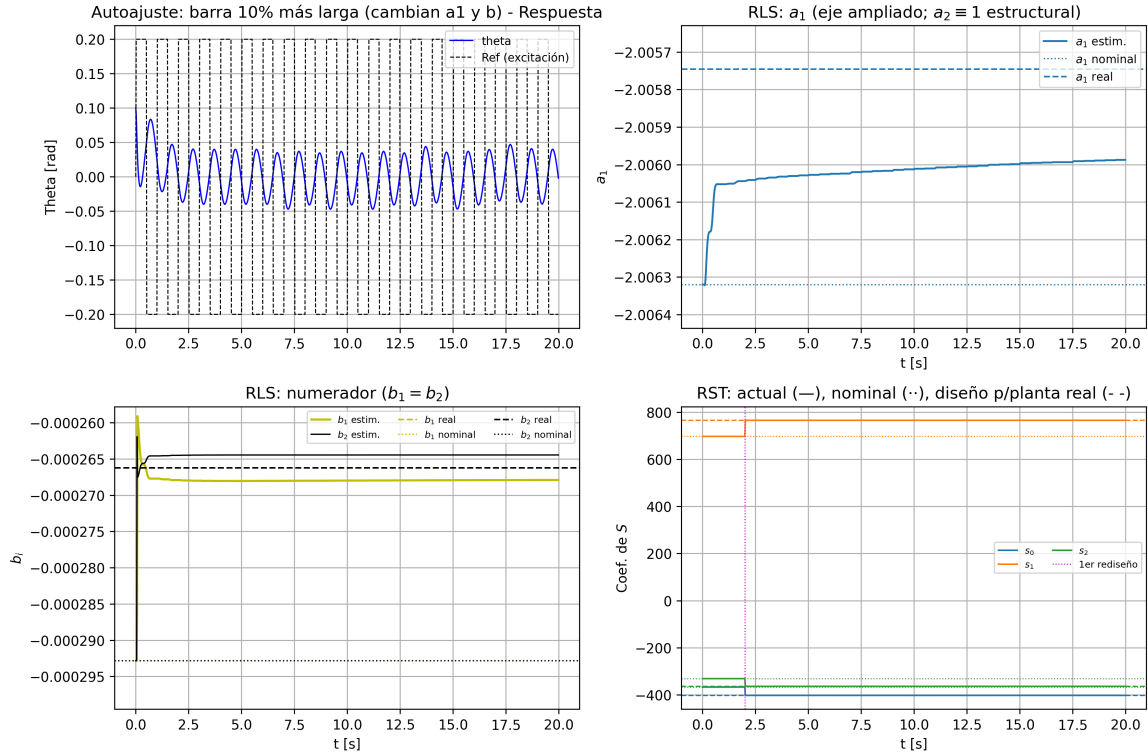


Figura 8: Autoajuste ante una barra 10 % más larga. A diferencia del caso de masa, ahora a_1 **sí se mueve** (eje ampliado) desde el prior nominal hacia la planta real, junto con b ; el regulador S converge al diseño de la planta real.

7.6. Suite de validación

El archivo `test_str_pendulo.py` automatiza la validación (todos los tests pasan):

- `test_diophantine_nominal`: el diseño nominal reproduce A_{lc} .
- `test_estabilidad_lazo`: $\max |z| < 1$ en los 4 escenarios y ambos modos.
- `test_theta_acotado`: $|\theta| \leq 0,12$ en todos los casos.
- `test_convergencia_RST_robusto` y `test_robusto_mejora_convergencia`: R, S, T convergen en robusto y mejoran respecto del básico en sinusoidal/ruido.
- `test_convergencia_RLS_robusto`: $\hat{\theta}$ cercano al modelo real cuando el prior coincide con la planta.
- `test_autoajuste_planta_distinta`: con la planta 50 % más pesada, la ganancia estimada converge al modelo *real* (no al prior), el S final difiere del nominal y el lazo es estable sobre la planta real.
- `test_autoajuste_longitud_mueve_a1`: con la barra 10 % más larga, el a_1 estimado se desplaza desde el nominal hacia el real y el lazo permanece estable.

8. Discusión

- **Persistencia de excitación.** Al ser una tarea de regulación ($\theta \rightarrow 0$), una vez estabilizado el sistema la excitación se desvanece. Por eso el RLS arranca desde el *prior* nominal y se congela la actualización con baja excitación: así el *certainty equivalence* se mantiene bien condicionado y estable. Iniciar el estimador desde un prior neutro provocó, en pruebas, deriva de parámetros y divergencia.
- **No cancelación de ceros.** El cero en $z \approx -1$ es no de fase mínima; cancelarlo produciría *ringing*. La colocación de polos sin cancelación evita este problema.

- **Comparación con el enfoque previo.** El STR discreto obtiene el regulador directamente en z resolviendo la Diofantina, eliminando la conversión a continuo por Tustin y la colocación de polos en s . El resultado es equivalente pero conceptualmente más directo y consistente con la implementación digital.

9. Conclusiones

Se implementó un STR indirecto en tiempo discreto que estabiliza la dinámica angular del péndulo invertido. El método combina estimación RLS on-line, resolución de la ecuación Diofantina por matriz de Sylvester (con acción integral y sin cancelar el cero no de fase mínima) y la ley de control RST. El diseño nominal reproduce exactamente los polos deseados y la variante robusta mantiene el desempeño acotado ante perturbaciones y ruido de medición, con mejoras claras en el caso de ruido (RMS de θ unas $6\times$ menor y margen de estabilidad preservado).

Debido a la pérdida de persistencia de excitación en la tarea de regulación y al uso de un prior nominal próximo a la planta, los cuatro escenarios principales demuestran sobre todo **estabilidad y robustez**. La *capacidad de autoajuste* propiamente dicha se validó por separado (Sección 7.5): con una planta 50 % más pesada y excitación persistente, el RLS se desplaza desde el prior nominal hacia el modelo real, el RST se rediseña a un controlador *distinto* del nominal y el lazo permanece estable, cumpliendo la definición de *self-tuning*. En un escenario con incertidumbre paramétrica mayor, el autoajuste requeriría relajar el gate de rediseño (que hoy sólo garantiza estabilidad frente al prior) e introducir excitación explícita.

Como material adicional se generan animaciones (.mp4) de cada escenario en `saved_runs_pid_autotuning/`.